



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

6

Engine Support Systems

Every game engine requires some low-level support systems that manage mundane but crucial tasks, such as starting up and shutting down the engine, configuring engine and game features, managing the engine's memory usage, handling access to file system(s), providing access to the wide range of heterogeneous asset types used by the game (meshes, textures, animations, audio, etc.), and providing debugging tools for use by the game development team. This chapter will focus on the lowest-level support systems found in most game engines. In the chapters that follow, we will explore some of the larger core systems, including resource management, human interface devices and in-game debugging tools.

6.1 Subsystem Start-Up and Shut-Down

A game engine is a complex piece of software consisting of many interacting subsystems. When the engine first starts up, each subsystem must be configured and initialized in a specific order. Interdependencies between subsystems implicitly define the order in which they must be started—i.e., if subsystem B depends on subsystem A, then A will need to be started up before B can be initialized. Shut-down typically occurs in the reverse order, so B would shut down first, followed by A.

6.1.1 C++ Static Initialization Order (or Lack Thereof)

Since the programming language used in most modern game engines is C++, we should briefly consider whether C++'s native start-up and shut-down semantics can be leveraged in order to start up and shut down our engine's subsystems. In C++, global and static objects are constructed before the program's entry point (`main()`, or `WinMain()` under Windows) is called. However, these constructors are called in a totally unpredictable order. The destructors of global and static class instances are called after `main()` (or `WinMain()`) returns, and once again they are called in an unpredictable order. Clearly this behavior is not desirable for initializing and shutting down the subsystems of a game engine, or indeed any software system that has interdependencies between its global objects.

This is somewhat unfortunate, because a common design pattern for implementing major subsystems such as the ones that make up a game engine is to define a *singleton class* (often called a *manager*) for each subsystem. If C++ gave us more control over the order in which global and static class instances were constructed and destroyed, we could define our singleton instances as globals, without the need for dynamic memory allocation. For example, we could write:

```
class RenderManager
{
public:
    RenderManager()
    {
        // start up the manager...
    }

    ~RenderManager()
    {
        // shut down the manager...
    }

    // ...
};

// singleton instance
static RenderManager gRenderManager;
```

Alas, with no way to directly control construction and destruction order, this approach won't work.

6.1.1.1 Construct On Demand

There is one C++ “trick” we can leverage here. A static variable that is declared within a function will not be constructed before `main()` is called, but rather on the first invocation of that function. So if our global singleton is function-static, we can control the order of construction for our global singletons.

```
class RenderManager
{
public:
    // Get the one and only instance.
    static RenderManager& get()
    {
        // This function-static will be constructed on the
        // first call to this function.
        static RenderManager sSingleton;
        return sSingleton;
    }

    RenderManager()
    {
        // Start up other managers we depend on, by
        // calling their get() functions first...
        VideoManager::get();
        TextureManager::get();

        // Now start up the render manager.
        // ...
    }

    ~RenderManager()
    {
        // Shut down the manager.
        // ...
    }
};
```

You’ll find that many software engineering textbooks suggest this design or a variant that involves dynamic allocation of the singleton as shown below.

```
static RenderManager& get()
{
    static RenderManager* gpSingleton = nullptr;
    if (gpSingleton == nullptr)
    {
        gpSingleton = new RenderManager;
```



```

    }
    ASSERT(gpSingleton);
    return *gpSingleton;
}

```

Unfortunately, this still gives us no way to control destruction order. It is possible that C++ will destroy one of the managers upon which the `RenderManager` depends for its shut-down procedure, prior to the `RenderManager`'s destructor being called. In addition, it's difficult to predict exactly when the `RenderManager` singleton will be constructed, because the construction will happen on the first call to `RenderManager::get()`—and who knows when that might be? Moreover, the programmers using the class may not be expecting an innocuous-looking `get()` function to do something expensive, like allocating and initializing a heavyweight singleton. This is an unpredictable and dangerous design. Therefore, we are prompted to resort to a more direct approach that gives us greater control.

6.1.2 A Simple Approach That Works

Let's presume that we want to stick with the idea of singleton managers for our subsystems. In this case, the simplest "brute-force" approach is to define explicit start-up and shut-down functions for each singleton manager class. These functions take the place of the constructor and destructor, and in fact we should arrange for the constructor and destructor to do *absolutely nothing*. That way, the start-up and shut-down functions can be explicitly called *in the required order* from within `main()` (or from some overarching singleton object that manages the engine as a whole). For example:

```

class RenderManager
{
public:
    RenderManager()
    {
        // do nothing
    }

    ~RenderManager()
    {
        // do nothing
    }

    void startUp()
    {
        // start up the manager...
    }
}

```

```

    }

    void shutDown()
    {
        // shut down the manager...
    }

    // ...
};

class PhysicsManager    { /* similar... */ };
class AnimationManager  { /* similar... */ };
class MemoryManager     { /* similar... */ };
class FileSystemManager { /* similar... */ };

// ...

RenderManager      gRenderManager;
PhysicsManager      gPhysicsManager;
AnimationManager    gAnimationManager;
TextureManager      gTextureManager;
VideoManager        gVideoManager;
MemoryManager       gMemoryManager;
FileSystemManager    gFileSystemManager;
// ...

int main(int argc, const char* argv)
{
    // Start up engine systems in the correct order.
    gMemoryManager.startUp();
    gFileSystemManager.startUp();
    gVideoManager.startUp();
    gTextureManager.startUp();
    gRenderManager.startUp();
    gAnimationManager.startUp();
    gPhysicsManager.startUp();
    // ...

    // Run the game.
    gSimulationManager.run();

    // Shut everything down, in reverse order.
    // ...
    gPhysicsManager.shutDown();

```

```
gAnimationManager.shutdown();  
gRenderManager.shutdown();  
gFileManager.shutdown();  
gMemoryManager.shutdown();  
  
return 0;  
}
```

There are “more elegant” ways to accomplish this. For example, you could have each manager register itself into a global priority queue and then walk this queue to start up all the managers in the proper order. You could define the manager-to-manager dependency graph by having each manager explicitly list the other managers upon which it depends and then write some code to calculate the optimal start-up order given their interdependencies. You could use the construct-on-demand approach outlined above. In my experience, the brute-force approach always wins out, because of the following:

- It’s simple and easy to implement.
- It’s explicit. You can see and understand the start-up order immediately by just looking at the code.
- It’s easy to debug and maintain. If something isn’t starting early enough, or is starting too early, you can just move one line of code.

One minor disadvantage to the brute-force manual start-up and shut-down method is that you might accidentally shut things down in an order that isn’t strictly the reverse of the start-up order. But I wouldn’t lose any sleep over it. As long as you can start up and shut down your engine’s subsystems successfully, you’re golden.

6.1.3 Some Examples from Real Engines

Let’s take a brief look at some examples of engine start-up and shut-down taken from real game engines.

6.1.3.1 OGRE

OGRE is by its authors’ admission a rendering engine, not a game engine per se. But by necessity it provides many of the low-level features found in full-fledged game engines, including a simple and elegant start-up and shut-down mechanism. Everything in OGRE is controlled by the singleton object `Ogre::Root`. It contains pointers to every other subsystem in OGRE and manages their creation and destruction. This makes it very easy for a programmer to start up OGRE—just new an instance of `Ogre::Root` and you’re done.

Here are a few excerpts from the OGRE source code so we can see what it's doing:

OgreRoot.h

```
class _OgreExport Root : public Singleton<Root>
{
    // <some code omitted...>

    // Singletons
    LogManager* mLogManager;
    ControllerManager* mControllerManager;
    SceneManagerEnumerator* mSceneManagerEnum;
    SceneManager* mCurrentSceneManager;
    DynLibManager* mDynLibManager;
    ArchiveManager* mArchiveManager;
    MaterialManager* mMaterialManager;
    MeshManager* mMeshManager;
    ParticleSystemManager* mParticleManager;
    SkeletonManager* mSkeletonManager;
    OverlayElementFactory* mPanelFactory;
    OverlayElementFactory* mBorderPanelFactory;
    OverlayElementFactory* mTextAreaFactory;
    OverlayManager* mOverlayManager;
    FontManager* mFontManager;
    ArchiveFactory *mZipArchiveFactory;
    ArchiveFactory *mFileSystemArchiveFactory;
    ResourceGroupManager* mResourceGroupManager;
    ResourceBackgroundQueue* mResourceBackgroundQueue;
    ShadowTextureManager* mShadowTextureManager;

    // etc.
};
```

OgreRoot.cpp

```
Root::Root(const String& pluginFileName,
           const String& configFileFileName,
           const String& logFileName) :
    mLogManager(0),
    mCurrentFrame(0),
    mFrameSmoothingTime(0.0f),
    mNextMovableObjectTypeFlag(1),
    mIsInitialised(false)
{
    // superclass will do singleton checking
    String msg;

    // Init
    mActiveRenderer = 0;
```

```

mVersion
    = StringConverter::toString(OGRE_VERSION_MAJOR)
    + "."
    + StringConverter::toString(OGRE_VERSION_MINOR)
    + "."
    + StringConverter::toString(OGRE_VERSION_PATCH)
    + OGRE_VERSION_SUFFIX + " "
    + "(" + OGRE_VERSION_NAME + ")";
mConfigFileName = configFileName;

// create log manager and default log file if there
// is no log manager yet
if(LogManager::getSingletonPtr() == 0)
{
    mLogManager = new LogManager();
    mLogManager->createLog(logFileName, true, true);
}

// dynamic library manager
mDynLibManager = new DynLibManager();
mArchiveManager = new ArchiveManager();

// ResourceGroupManager
mResourceGroupManager = new ResourceGroupManager();

// ResourceBackgroundQueue
mResourceBackgroundQueue
    = new ResourceBackgroundQueue();

// and so on...
}

```

OGRE provides a templated `Ogre::Singleton` base class from which all of its singleton (manager) classes derive. If you look at its implementation, you'll see that `Ogre::Singleton` does not use deferred construction but instead relies on `Ogre::Root` to explicitly new each singleton. As we discussed above, this is done to ensure that the singletons are created and destroyed in a well-defined order.

6.1.3.2 Naughty Dog's *Uncharted* and *The Last of Us* Series

The engine created by Naughty Dog, Inc. for its *Uncharted* and *The Last of Us* series of games uses a similar explicit technique for starting up its subsystems. You'll notice by looking at the following code that engine start-up is not always a simple sequence of allocating singleton instances. A wide range of operating system services, third-party libraries and so on must all be started

up during engine initialization. Also, dynamic memory allocation is avoided wherever possible, so many of the singletons are statically allocated objects (e.g., `g_fileSystem`, `g_languageMgr`, etc.) It's not always pretty, but it gets the job done.

```
Err BigInit()
{
    init_exception_handler();

    U8* pPhysicsHeap = new(kAllocGlobal, kAlign16)
        U8[ALLOCATION_GLOBAL_PHYS_HEAP];
    PhysicsAllocatorInit(pPhysicsHeap,
        ALLOCATION_GLOBAL_PHYS_HEAP);

    g_textDb.Init();
    g_textSubDb.Init();
    g_spuMgr.Init();

    g_drawScript.InitPlatform();

    PlatformUpdate();

    thread_t init_thr;
    thread_create(&init_thr, threadInit, 0, 30,
        64*1024, 0, "Init");

    char masterConfigFileName[256];
    snprintf(masterConfigFileName,
        sizeof(masterConfigFileName),
        MASTER_CFG_PATH);
    {
        Err err = ReadConfigFromFile(
            masterConfigFileName);
        if (err.Failed())
        {
            MsgErr("Config file not found (%s).\n",
                masterConfigFileName);
        }
    }

    memset(&g_discInfo, 0, sizeof(BootDiscInfo));
    int err1 = GetBootDiscInfo(&g_discInfo);
    Msg("GetBootDiscInfo() : 0x%x\n", err1);
    if(err1 == BOOTDISCINFO_RET_OK)
    {
        printf("titleId      : [%s]\n",
            g_discInfo.titleId);
    }
}
```

```

        printf("parentalLevel : [%d]\n",
               g_discInfo.parentalLevel);
    }

    g_fileSystem.Init(g_gameInfo.m_onDisc);

    g_languageMgr.Init();
    if (g_shouldQuit) return Err::kOK;

    // and so on...

```

6.2 Memory Management

As game developers, we are always trying to make our code run more quickly. The performance of any piece of software is dictated not only by the algorithms it employs, or the efficiency with which those algorithms are coded, but also by how the program *utilizes memory* (RAM). Memory affects performance in two ways:

1. *Dynamic memory allocation* via `malloc()` or C++'s global operator `new` is a very slow operation. We can improve the performance of our code by either avoiding dynamic allocation altogether or by making use of custom memory allocators that greatly reduce allocation costs.
2. On modern CPUs, the performance of a piece of software is often dominated by its *memory access patterns*. As we'll see, data that is located in small, *contiguous* blocks of memory can be operated on much more efficiently by the CPU than if that same data were to be spread out across a wide range of memory addresses. Even the most efficient algorithm, coded with the utmost care, can be brought to its knees if the data upon which it operates is not laid out efficiently in memory.

In this section, we'll learn how to optimize our code's memory utilization along these two axes.

6.2.1 Optimizing Dynamic Memory Allocation

Dynamic memory allocation via `malloc()` and `free()` or C++'s global `new` and `delete` operators—also known as *heap allocation*—is typically very slow. The high cost can be attributed to two main factors. First, a heap allocator is a general-purpose facility, so it must be written to handle any allocation size, from one byte to one gigabyte. This requires a lot of management overhead, making the `malloc()` and `free()` functions inherently slow. Second,

on most operating systems a call to `malloc()` or `free()` must first context-switch from user mode into kernel mode, process the request and then context-switch back to the program. These context switches can be extraordinarily expensive. One rule of thumb often followed in game development is:

Keep heap allocations to a minimum, and never allocate from the heap within a tight loop.

Of course, no game engine can entirely avoid dynamic memory allocation, so most game engines implement one or more custom allocators. A custom allocator can have better performance characteristics than the operating system's heap allocator for two reasons. First, a custom allocator can satisfy requests from a preallocated memory block (itself allocated using `malloc()` or `new`, or declared as a global variable). This allows it to run in user mode and entirely avoid the cost of context-switching into the operating system. Second, by making various assumptions about its usage patterns, a custom allocator can be much more efficient than a general-purpose heap allocator.

In the following sections, we'll take a look at some common kinds of custom allocators. For additional information on this topic, see Christian Gyrling's excellent blog post, <http://www.swedishcoding.com/2008/08/31/are-we-out-of-memory>.

6.2.1.1 Stack-Based Allocators

Many games allocate memory in a stack-like fashion. Whenever a new game level is loaded, memory is allocated for it. Once the level has been loaded, little or no dynamic memory allocation takes place. At the conclusion of the level, its data is unloaded and all of its memory can be freed. It makes a lot of sense to use a stack-like data structure for these kinds of memory allocations.

A *stack allocator* is very easy to implement. We simply allocate a large contiguous block of memory using `malloc()` or global `new`, or by declaring a global array of bytes (in which case the memory is effectively allocated out of the executable's BSS segment). A pointer to the top of the stack is maintained. All memory addresses below this pointer are considered to be in use, and all addresses above it are considered to be free. The top pointer is initialized to the lowest memory address in the stack. Each allocation request simply moves the pointer up by the requested number of bytes. The most recently allocated block can be freed by simply moving the top pointer back down by the size of the block.

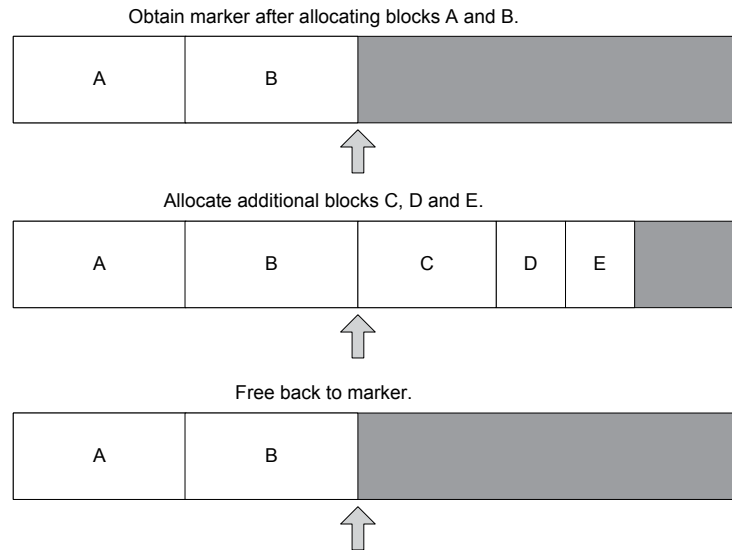


Figure 6.1. Stack allocation and freeing back to a marker.

It is important to realize that with a stack allocator, memory cannot be freed in an arbitrary order. All frees must be performed in an order opposite to that in which they were allocated. One simple way to enforce these restrictions is to disallow individual blocks from being freed at all. Instead, we can provide a function that rolls the stack top back to a previously marked location, thereby freeing all blocks between the current top and the roll-back point.

It's important to always roll the top pointer back to a point that lies at the boundary between two allocated blocks, because otherwise new allocations would overwrite the tail end of the top-most block. To ensure that this is done properly, a stack allocator often provides a function that returns a *marker* representing the current top of the stack. The roll-back function then takes one of these markers as its argument. This is depicted in Figure 6.1. The interface of a stack allocator often looks something like this.

```
class StackAllocator
{
public:
    // Stack marker: Represents the current top of the
    // stack. You can only roll back to a marker, not to
    // arbitrary locations within the stack.
    typedef U32 Marker;
```

```

    // Constructs a stack allocator with the given total
    // size.
    explicit StackAllocator(U32 stackSize_bytes);

    // Allocates a new block of the given size from stack
    // top.
    void* alloc(U32 size_bytes);

    // Returns a marker to the current stack top.
    Marker getMarker();

    // Rolls the stack back to a previous marker.
    void freeToMarker(Marker marker);

    // Clears the entire stack (rolls the stack back to
    // zero).
    void clear();

private:
    // ...
};

```

Double-Ended Stack Allocators

A single memory block can actually contain two stack allocators—one that allocates up from the bottom of the block and one that allocates down from the top of the block. A double-ended stack allocator is useful because it uses memory more efficiently by allowing a trade-off to occur between the memory usage of the bottom stack and the memory usage of the top stack. In some situations, both stacks may use roughly the same amount of memory and meet in the middle of the block. In other situations, one of the two stacks may eat up a lot more memory than the other stack, but all allocation requests can still be satisfied as long as the total amount of memory requested is not larger than the block shared by the two stacks. This is depicted in Figure 6.2.

In Midway’s *Hydro Thunder* arcade game, all memory allocations are made from a single large block of memory managed by a double-ended stack allocator. The bottom stack is used for loading and unloading levels (race tracks), while the top stack is used for temporary memory blocks that are allocated and freed every frame. This allocation scheme worked extremely well and ensured that *Hydro Thunder* never suffered from memory fragmentation problems (see Section 6.2.1.4). Steve Ranck, *Hydro Thunder*’s lead engineer, describes this allocation technique in depth in [8, Section 1.9].



Figure 6.2. A double-ended stack allocator.

6.2.1.2 Pool Allocators

It's quite common in game engine programming (and software engineering in general) to allocate lots of small blocks of memory, each of which are the same size. For example, we might want to allocate and free matrices, or iterators, or links in a linked list, or renderable mesh instances. For this type of memory allocation pattern, a *pool allocator* is often the perfect choice.

A pool allocator works by preallocating a large block of memory whose size is an exact multiple of the size of the elements that will be allocated. For example, a pool of 4×4 matrices would be an exact multiple of 64 bytes—that's 16 elements per matrix times four bytes per element (assuming each element is a 32-bit `float`). Each element within the pool is added to a linked list of free elements; when the pool is first initialized, the free list contains all of the elements. Whenever an allocation request is made, we simply grab the next free element off the free list and return it. When an element is freed, we simply tack it back onto the free list. Both allocations and frees are $O(1)$ operations, since each involves only a couple of pointer manipulations, no matter how many elements are currently free. (The notation $O(1)$ is an example of “big O” notation. In this case it means that the execution times of both allocations and frees are roughly constant and do not depend on things like the number of elements currently in the pool. See Section 6.3.3 for an explanation of “big O” notation.)

The linked list of free elements can be a singly-linked list, meaning that we need a single pointer (four bytes on 32-bit machines or eight bytes on 64-bit machines) for each free element. Where should we obtain the memory for all these pointers? Certainly they could be stored in a separate preallocated memory block, occupying `(sizeof(void*) * numElementsInPool)` bytes. However, this is unduly wasteful. The key is to realize that the memory blocks residing on the free list are, by definition, free memory blocks. So why not store each free list “next” pointer *within the free block itself*? This little “trick” works as long as `elementSize >= sizeof(void*)`. We don't waste any memory, because our free list pointers all reside inside the free memory blocks—in memory that wasn't being used for anything anyway!

If each element is smaller than a pointer, then we can use pool element in-

dices instead of pointers to implement our linked list. For example, if our pool contains 16-bit integers, then we can use 16-bit indices as the “next pointers” in our linked list. This works as long as the pool doesn’t contain more than $2^{16} = 65,536$ elements.

6.2.1.3 Aligned Allocations

As we saw in Section 3.3.7.1, every variable and data object has an alignment requirement. An 8-bit integer variable can be aligned to any address, but a 32-bit integer or floating-point variable must be 4-byte aligned, meaning its address can only end in the nibbles 0x0, 0x4, 0x8 or 0xC. A 128-bit SIMD vector value generally has a 16-byte alignment requirement, meaning that its memory address can end only in the nibble 0x0. On the PS3, memory blocks that are to be transferred to an SPU via the direct memory access (DMA) controller should be 128-byte aligned for maximum DMA throughput, meaning they can only end in the bytes 0x00 or 0x80.

All memory allocators must be capable of returning aligned memory blocks. This is relatively straightforward to implement. We simply allocate a little bit more memory than was actually requested, shift the address of the memory block upward slightly so that it is aligned properly, and then return the shifted address. Because we allocated a bit more memory than was requested, the returned block will still be large enough, even with the slight upward shift.

In most implementations, the number of additional bytes allocated is equal to the alignment minus one, which is the worst-case alignment shift we can make. For example, if we want a 16-byte aligned memory block, the worst case would be to get back an unaligned pointer that ends in 0x1, because it would require us to apply a shift of 15 bytes to bring it to a 16-byte boundary.

Here’s one possible implementation of an aligned memory allocator:

```
// Shift the given address upwards if/as necessary to
// ensure it is aligned to the given number of bytes.
inline uintptr_t AlignAddress(uintptr_t addr, size_t align)
{
    const size_t mask = align - 1;
    assert((align & mask) == 0); // pwr of 2
    return (addr + mask) & ~mask;
}

// Shift the given pointer upwards if/as necessary to
// ensure it is aligned to the given number of bytes.
template<typename T>
inline T* AlignPointer(T* ptr, size_t align)
```

```

{
    const uintptr_t addr = reinterpret_cast<uintptr_t>(ptr);
    const uintptr_t addrAligned = AlignAddress(addr, align);
    return reinterpret_cast<T*>(addrAligned);
}

// Aligned allocation function. IMPORTANT: 'align'
// must be a power of 2 (typically 4, 8 or 16).
void* AllocAligned(size_t bytes, size_t align)
{
    // Determine worst case number of bytes we'll need.
    size_t worstCaseBytes = bytes + align - 1;

    // Allocate unaligned block.
    U8* pRawMem = new U8[worstCaseBytes];

    // Align the block.
    return AlignPointer(pRawMem, align);
}

```

The alignment “magic” is performed by the function `AlignAddress()`. Here’s how it works: Given an address and a desired alignment L , we can align that address to an L -byte boundary by first adding $L - 1$ to it, and then stripping off the N least-significant bits of the resulting address, where $N = \log_2(L)$. For example, to align any address to a 16-byte boundary, we shift it up by 15 bytes and then mask off the $N = \log_2(16) = 4$ least-significant bits.

To strip off these bits, we need a *mask* that we can apply to the address using the bitwise AND operator. Because L is always a power of two, $L - 1$ will be a mask with binary 1s in the N least-significant bits and binary 0s everywhere else. So all we need to do is *invert* this mask and then AND it with the address (`addr & ~mask`).

Freeing Aligned Blocks

When an aligned block is later freed, we will be passed the *shifted* address, not the original address that we allocated. But in order to free the memory, we need to free the address that was actually returned by `new`. How can we convert an aligned address back into its original, unaligned address?

One simple approach is to store the shift (i.e., the difference between the aligned address and the original address) some place where our free function will be able to find it. Recall that we actually allocate `align - 1` extra bytes in `AllocAligned()`, in order to give us some room to align the pointer. Those

extra bytes are a perfect place to store the shift value. The smallest shift we'll ever make is one byte, so that's the minimum space we'll have to store the offset. Therefore, given an aligned pointer p , we can simply store the shift as a one-byte value at address $p - 1$.

However, there's one problem: It's possible that the raw address returned by `new` will already be aligned. In that case, the code we presented above would not shift the raw address at all, meaning there'd be no extra bytes into which to store the offset. To overcome this, we simply allocate L extra bytes, instead of $L - 1$, and then we *always* shift the raw pointer up to the next L -byte boundary, even if it was already aligned. Now the maximum shift will be L bytes, and the minimum shift will be 1 byte. So there will always be at least one extra byte into which we can store our shift value.

Storing the shift in a single byte works for alignments up to and including 128. We never ever shift a pointer by zero bytes, so we can make this scheme work for up to 256-byte alignment by interpreting the impossible shift value of zero as a 256-byte shift. (For larger alignments, we'd have to allocate even more bytes, and shift the pointer even further up, to make room for a wider "header".)

Here's how the modified `AllocAligned()` function and its corresponding `FreeAligned()` function could be implemented. The process of allocating and freeing aligned blocks is illustrated in Figure 6.3.

```
// Aligned allocation function. IMPORTANT: 'align'
// must be a power of 2 (typically 4, 8 or 16).
void* AllocAligned(size_t bytes, size_t align)
{
    // Allocate 'align' more bytes than we need.
    size_t actualBytes = bytes + align;

    // Allocate unaligned block.
    U8* pRawMem = new U8[actualBytes];

    // Align the block. If no alignment occurred,
    // shift it up the full 'align' bytes so we
    // always have room to store the shift.
    U8* pAlignedMem = AlignPointer(pRawMem, align);
    if (pAlignedMem == pRawMem)
        pAlignedMem += align;

    // Determine the shift, and store it.
    // (This works for up to 256-byte alignment.)
    ptrdiff_t shift = pAlignedMem - pRawMem;
    assert(shift > 0 && shift <= 256);
```

```

    pAlignedMem[-1] = static_cast<U8>(shift & 0xFF);

    return pAlignedMem;
}

void FreeAligned(void* pMem)
{
    if (pMem)
    {
        // Convert to U8 pointer.
        U8* pAlignedMem = reinterpret_cast<U8*>(pMem);

        // Extract the shift.
        ptrdiff_t shift = pAlignedMem[-1];
        if (shift == 0)
            shift = 256;

        // Back up to the actual allocated address,
        // and array-delete it.
        U8* pRawMem = pAlignedMem - shift;
        delete[] pRawMem;
    }
}

```

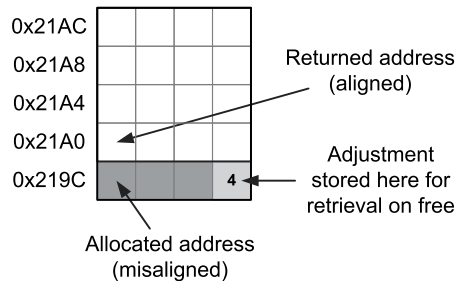


Figure 6.3. Aligned memory allocation with a 16-byte alignment requirement. The difference between the allocated memory address and the adjusted (aligned) address is stored in the byte immediately preceding the adjusted address, so that it may be retrieved during free.

6.2.1.4 Single-Frame and Double-Buffered Memory Allocators

Virtually all game engines allocate at least some temporary data during the game loop. This data is either discarded at the end of each iteration of the loop or used on the next frame and then discarded. This allocation pattern is so common that many engines support *single-frame* and *double-buffered* allocators.

Single-Frame Allocators

A single-frame allocator is implemented by reserving a block of memory and managing it with a simple stack allocator as described above. At the beginning of each frame, the stack's "top" pointer is cleared to the bottom of the memory block. Allocations made during the frame grow toward the top of the block. Rinse and repeat.

```
StackAllocator g_singleFrameAllocator;

// Main Game Loop
while (true)
{
    // Clear the single-frame allocator's buffer every
    // frame.
    g_singleFrameAllocator.clear();

    // ...

    // Allocate from the single-frame buffer. We never
    // need to free this data! Just be sure to use it
    // only this frame.
    void* p = g_singleFrameAllocator.alloc(nBytes);

    // ...
}
```

One of the primary benefits of a single-frame allocator is that allocated memory needn't ever be freed—we can rely on the fact that the allocator will be cleared at the start of every frame. Single-frame allocators are also blindingly fast. The one big negative is that using a single-frame allocator requires a reasonable level of discipline on the part of the programmer. You need to realize that a memory block allocated out of the single-frame buffer will only be valid during the current frame. Programmers must *never* cache a pointer to a single-frame memory block across the frame boundary!

Double-Buffered Allocators

A double-buffered allocator allows a block of memory allocated on frame i to be used on frame $(i + 1)$. To accomplish this, we create two single-frame stack allocators of equal size and then ping-pong between them every frame.

```
class DoubleBufferedAllocator
{
    U32          m_curStack;
```



```

        StackAllocator m_stack[2];

public:

    void swapBuffers()
    {
        m_curStack = (U32)!m_curStack;
    }

    void clearCurrentBuffer()
    {
        m_stack[m_curStack].clear();
    }

    void* alloc(U32 nBytes)
    {
        return m_stack[m_curStack].alloc(nBytes);
    }

    // ...
};

// ...

DoubleBufferedAllocator g_doubleBufAllocator;

// Main Game Loop
while (true)
{
    // Clear the single-frame allocator every frame as
    // before.
    g_singleFrameAllocator.clear();

    // Swap the active and inactive buffers of the double-
    // buffered allocator.
    g_doubleBufAllocator.swapBuffers();

    // Now clear the newly active buffer, leaving last
    // frame's buffer intact.
    g_doubleBufAllocator.clearCurrentBuffer();

    // ...

    // Allocate out of the current buffer, without
    // disturbing last frame's data. Only use this data
    // this frame or next frame. Again, this memory never

```

```
// needs to be freed.  
void* p = g_doubleBufAllocator.alloc(nBytes);  
  
// ...  
}
```

This kind of allocator is extremely useful for caching the results of asynchronous processing on a multicore game console like the Xbox 360, Xbox One, PlayStation 3 or PlayStation 4. On frame i , we can kick off an asynchronous job on one of the PS4's cores, for example, handing it the address of a destination buffer that has been allocated from our double-buffered allocator. The job runs and produces its results some time before the end of frame i , storing them into the buffer we provided. On frame $(i + 1)$, the buffers are swapped. The results of the job are now in the inactive buffer, so they will not be overwritten by any double-buffered allocations that might be made during this frame. As long as we use the results of the job before frame $(i + 2)$, our data won't be overwritten.

6.2.2 Memory Fragmentation

Another problem with dynamic heap allocations is that memory can become *fragmented* over time. When a program first runs, its heap memory is entirely free. When a block is allocated, a contiguous region of heap memory of the appropriate size is marked as “in use,” and the remainder of the heap remains free. When a block is freed, it is marked as such, and adjacent free blocks are merged into a single, larger free block. Over time, as allocations and deallocations of various sizes occur in random order, the heap memory begins to look like a patchwork of free and used blocks. We can think of the free regions as “holes” in the fabric of used memory. When the number of holes becomes large, and/or the holes are all relatively small, we say the memory has become *fragmented*. This is illustrated in Figure 6.4.

The problem with memory fragmentation is that allocations may fail even when there are enough free bytes to satisfy the request. The crux of the problem is that allocated memory blocks must always be *contiguous*. For example, in order to satisfy a request of 128 KiB, there must exist a free “hole” that is 128 KiB or larger. If there are two holes, each of which is 64 KiB in size, then enough bytes are available but the allocation fails because they are not *contiguous bytes*.

Memory fragmentation is not as much of a problem on operating systems that support *virtual memory*. A virtual memory system maps discontinuous blocks of physical memory known as *pages* into a *virtual address space*, in

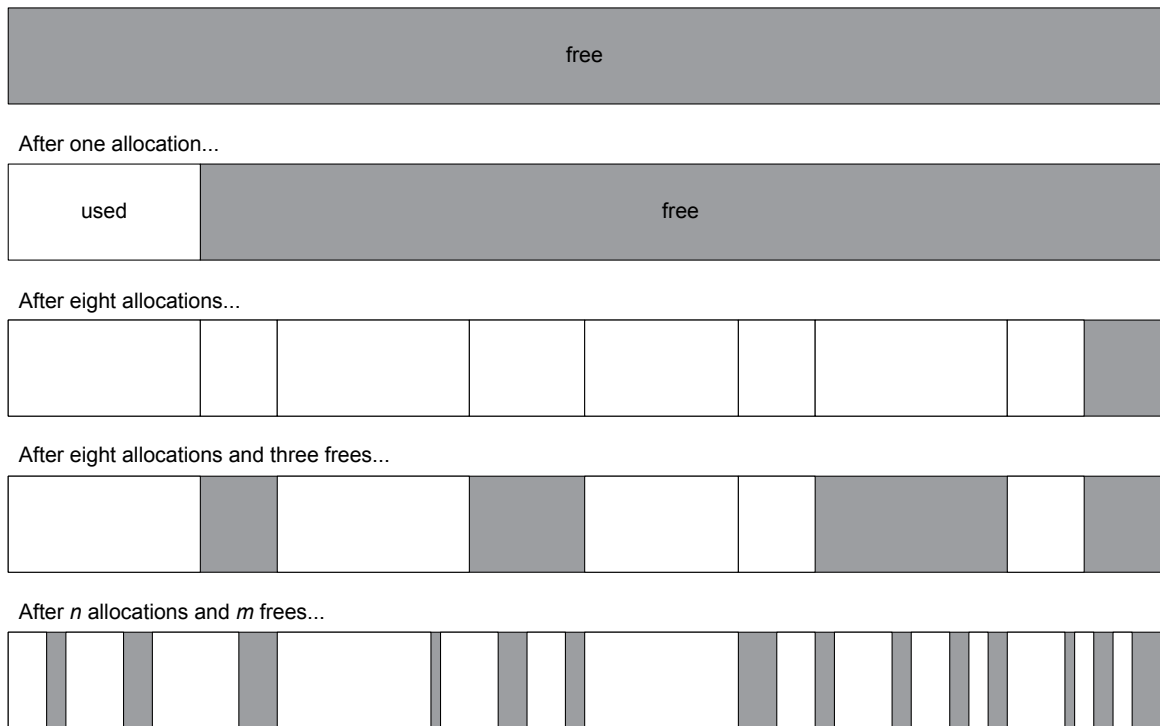


Figure 6.4. Memory fragmentation.

which the pages appear to the application to be contiguous. Stale pages can be swapped to the hard disk when physical memory is in short supply and reloaded from disk when they are needed. For a detailed discussion of how virtual memory works, see http://en.wikipedia.org/wiki/Virtual_memory. Most embedded systems cannot afford to implement a virtual memory system. While some modern consoles do technically support it, most console game engines still do not make use of virtual memory due to the inherent performance overhead.

6.2.2.1 Avoiding Fragmentation with Stack and Pool Allocators

The detrimental effects of memory fragmentation can be avoided by using stack and/or pool allocators.

- A stack allocator is impervious to fragmentation because allocations are always contiguous, and blocks must be freed in an order opposite to that in which they were allocated. This is illustrated in Figure 6.5.

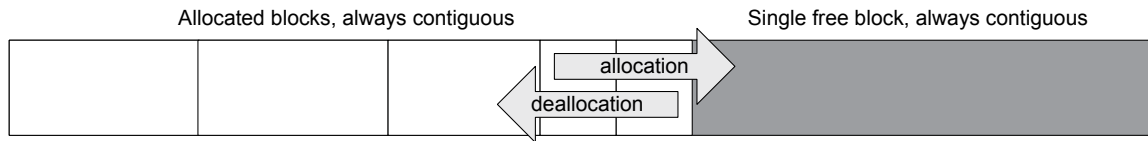


Figure 6.5. A stack allocator is free from fragmentation problems.

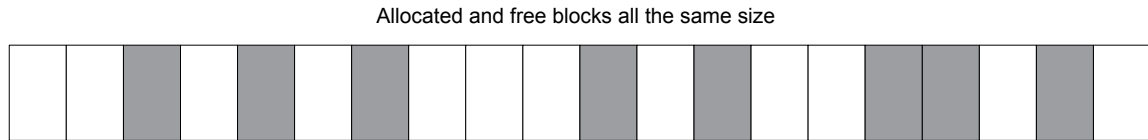


Figure 6.6. A pool allocator is not degraded by fragmentation.

- A pool allocator is also free from fragmentation problems. Pools *do* become fragmented, but the fragmentation never causes premature out-of-memory conditions as it does in a general-purpose heap. Pool allocation requests can never fail due to a lack of a large enough contiguous free block, because all of the blocks are exactly the same size. This is shown in Figure 6.6.

6.2.2.2 Defragmentation and Relocation

When differently sized objects are being allocated and freed in a random order, neither a stack-based allocator nor a pool-based allocator can be used. In such cases, fragmentation can be avoided by periodically *defragmenting* the heap. Defragmentation involves coalescing all of the free “holes” in the heap by shifting allocated blocks from higher memory addresses down to lower addresses (thereby shifting the holes up to higher addresses). One simple algorithm is to search for the first “hole” and then take the allocated block immediately above the hole and shift it down to the start of the hole. This has the effect of “bubbling up” the hole to a higher memory address. If this process is repeated, eventually all the allocated blocks will occupy a contiguous region of memory at the low end of the heap’s address space, and all the holes will have bubbled up into one big hole at the high end of the heap. This is illustrated in Figure 6.7.

The shifting of memory blocks described above is not particularly tricky to implement. What *is* tricky is accounting for the fact that we’re moving *allocated* blocks of memory around. If anyone has a *pointer* into one of these allocated blocks, then moving the block will invalidate the pointer.

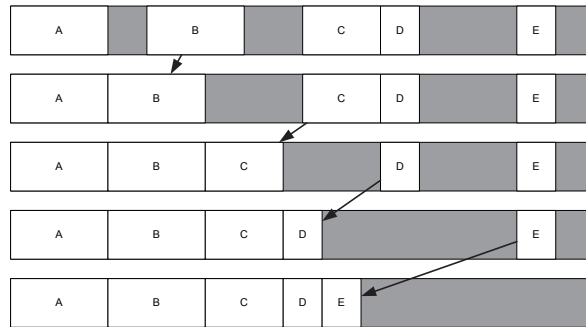


Figure 6.7. Defragmentation by shifting allocated blocks to lower addresses.

The solution to this problem is to patch any and all pointers into a shifted memory block so that they point to the correct new address after the shift. This procedure is known as *pointer relocation*. Unfortunately, there is no general-purpose way to *find* all the pointers that point into a particular region of memory. So if we are going to support memory defragmentation in our game engine, programmers must either carefully keep track of all the pointers manually so they can be relocated, or pointers must be abandoned in favor of something inherently more amenable to relocation, such as *smart pointers* or *handles*.

A smart pointer is a small class that contains a pointer and acts like a pointer for most intents and purposes. But because a smart pointer is a class, it can be coded to handle memory relocation properly. One approach is to arrange for all smart pointers to add themselves to a global linked list. Whenever a block of memory is shifted within the heap, the linked list of all smart pointers can be scanned, and each pointer that points into the shifted block of memory can be adjusted appropriately.

A handle is usually implemented as an index into a non-relocatable table, which itself contains the pointers. When an allocated block is shifted in memory, the handle table can be scanned and all relevant pointers found and updated automatically. Because the handles are just indices into the pointer table, their values never change no matter how the memory blocks are shifted, so the objects that use the handles are never affected by memory relocation.

Another problem with relocation arises when certain memory blocks cannot be relocated. For example, if you are using a third-party library that does not use smart pointers or handles, it's possible that any pointers into its data structures will not be relocatable. The best way around this problem is usually to arrange for the library in question to allocate its memory from a special buffer outside of the relocatable memory area. The other option is to simply

accept that some blocks will not be relocatable. If the number and size of the non-relocatable blocks are both small, a relocation system will still perform quite well.

It is interesting to note that all of Naughty Dog's engines have supported defragmentation. Handles are used wherever possible to avoid the need to relocate pointers. However, in some cases raw pointers cannot be avoided. These pointers are carefully tracked and relocated manually whenever a memory block is shifted due to defragmentation. A few of Naughty Dog's game object classes are not relocatable for various reasons. However, as mentioned above, this doesn't pose any practical problems, because the number of such objects is always very small, and their sizes are tiny when compared to the overall size of the relocatable memory area.

Amortizing Defragmentation Costs

Defragmentation can be a slow operation because it involves copying memory blocks. However, we needn't fully defragment the heap all at once. Instead, the cost can be amortized over many frames. We can allow up to N allocated blocks to be shifted each frame, for some small value of N like 8 or 16. If our game is running at 30 frames per second, then each frame lasts $1/30$ of a second (33 ms). So, the heap can usually be completely defragmented in less than one second without having any noticeable effect on the game's frame rate. As long as allocations and deallocations aren't happening at a faster rate than the defragmentation shifts, the heap will remain mostly defragmented at all times.

This approach is only valid when the size of each block is relatively small, so that the time required to move a single block does not exceed the time allotted to relocation each frame. If very large blocks need to be relocated, we can often break them up into two or more subblocks, each of which can be relocated independently. This hasn't proved to be a problem in Naughty Dog's engine, because relocation is only used for dynamic game objects, and they are never larger than a few kibibytes—and usually much smaller.

6.3 Containers

Game programmers employ a wide variety of collection-oriented data structures, also known as *containers* or *collections*. The job of a container is always the same—to house and manage zero or more data elements; however, the details of how they do this vary greatly, and each type of container has its pros and cons. Common container data types include, but are certainly not limited